RETAIL SALES ANALYTICS PROJECT

Interview Preparation & Mentor Q&A Guide

A private study companion — for practicing how to explain this project confidently, in your own words

---

Prepared for

Bijoy Mistry

Companion to: Retail Sales Analytics — Business Intelligence Case Study

How to Use This Guide

This document is structured as rehearsal material, not a script to memorize word-for-word. Read each answer, then close the document and say it out loud in your own words — a memorized answer reads as memorized the moment a mentor or interviewer asks a follow-up. The goal is to internalize the reasoning behind each decision well enough that you could defend it under a slightly different question.

The guide is organized in the order a real conversation tends to flow: an opening pitch, then business framing, then progressively more technical questions, then tougher/skeptical challenge questions a mentor is likely to throw at you, and finally behavioral questions.

1. The 60-Second Project Pitch

Every conversation about this project should be able to start here. Practice this until it feels natural, not recited.

Q: Walk me through this project in a minute or two.

A: I built an end-to-end retail sales analytics solution in Power BI for a retail dataset covering customers, orders, order-level transactions, and products. The core problem was that reporting was manual and fragmented — no single source of truth for sales performance. I designed a star schema data model, built a centralized DAX measure library for KPIs like Total Sales, Total Orders, and Average Order Value, and then designed a single-page executive dashboard so leadership could see trend, regional, category, and payment-mode performance and drill into any of it interactively. The whole thing is documented like a client engagement — business requirements, data dictionary, model rationale, and business recommendations — because I wanted it to reflect how I'd actually deliver this at a company, not just complete an exercise.

2. Business & Domain Questions

Q: Why does this dashboard matter to a business, in plain terms?

A: Because it replaces manual spreadsheet reporting with a live, self-service view. Instead of an analyst pulling numbers every week, any stakeholder can filter by year, category, or region themselves and get a trustworthy answer instantly — and everyone is looking at the same numbers, because they all come from the same measures.

Q: How did you decide which KPIs to include?

A: I mapped KPIs to the stakeholders who'd actually use them — I framed it as a lightweight BRD. The CEO needs a top-line health check (Total Sales), the Sales Manager needs volume (Total Orders), Marketing needs reach (Total Customers), Operations needs demand signals (Quantity Sold), and Finance needs efficiency (Average Order Value). I didn't just build every metric I could compute — I built the ones tied to a specific decision someone needed to make.

Q: What insight from the dashboard would you act on first if you were the business owner?

A: The category mix is fairly balanced — Furniture leads at around 27%, with Grocery and Clothing close behind — so there's no single category carrying the business, which is a healthy sign. What I'd actually chase first is the regional gap: one region is visibly underperforming the others, and I'd want a root-cause review — pricing, logistics, or assortment — before pulling any budget from it.

Q: If this dashboard had to convince a skeptical stakeholder to adopt it over their existing spreadsheet, what's the strongest argument?

A: Consistency and speed. A spreadsheet answer depends on whoever built the pivot table that week; this dashboard's numbers are governed by one measure library, so 'Total Sales' means the same thing every time, for every stakeholder, and updates the moment a filter changes instead of requiring a re-export.

3. Data Modeling & Star Schema Questions

Q: Why did you choose a star schema instead of a single flat table or a snowflake schema?

A: A flat table would duplicate customer and product attributes on every transaction row, which bloats the model and makes one-to-many relationships awkward to express. A snowflake schema — normalizing dimensions further — adds join complexity without a real benefit here, since none of my dimensions have repeating attribute groups that need their own sub-tables. Star schema gave me the best balance: simple one-to-many relationships, predictable filter propagation, and DAX that stays easy to reason about.

Q: Walk me through your relationships and their cardinality.

A: Customers to Orders is one-to-many on CustomerID, Orders to Order_Details is one-to-many on OrderID, Products to Order_Details is one-to-many on ProductID, and Calendar to Orders is one-to-many on the date field. All of them are single-direction cross-filters, flowing from dimension to fact — I avoided bidirectional relationships because they can create ambiguous filter paths, especially once more tables get added.

Q: What's the difference between a fact table and a dimension table in your model?

A: Order_Details is my fact table — it holds the actual transactional measures: quantity, price, discount, net sales. Customers, Products, and Calendar are dimension tables — they describe who, what, and when, and they're what the slicers filter by. Orders sits in between as an order header, which is common in real retail schemas where a fact table doesn't have to hold every descriptive attribute directly.

Q: Why keep a separate, disconnected KPI Measures table instead of putting measures on the fact table?

A: It's purely organizational — DAX measures aren't affected by which table they're 'homed' to, so I used an empty table just to give the measure library a clean, obvious home in the field list. It makes the model much easier for someone else to navigate, especially non-technical stakeholders who don't need to see it mixed in with raw columns.

Q: How would this model change if the data volume grew to millions of rows?

A: I'd look at incremental refresh so Power BI only reprocesses recent partitions instead of the full history each time, consider aggregation tables for the heaviest visuals, and double check that all my relationship columns are as low-cardinality as possible, since VertiPaq compresses better that way. I'd also move from a static extract to DirectQuery or a proper scheduled refresh against the source database.

4. DAX Technical Questions

Q: Explain the difference between a calculated column and a measure.

A: A calculated column is computed row-by-row at refresh time and stored in the model, taking up memory — it has row context but no awareness of the visual's filters. A measure is computed at query time in whatever filter context the visual or slicer creates, so it's dynamic and doesn't bloat the model. Almost everything in my KPI library is a measure, because I want it to respond to slicers, not be fixed at refresh time.

Q: What does CALCULATE actually do, conceptually?

A: CALCULATE evaluates an expression inside a modified filter context — it's how DAX changes what's being filtered before aggregating. It's also the mechanism behind context transition: when CALCULATE is used inside a row context, like inside a calculated column or SUMX, it converts the current row into an equivalent filter, so aggregate functions inside it behave as if a filter equal to that row's values had been applied.

Q: Why DIVIDE() instead of the '/' operator?

A: DIVIDE() returns BLANK() — or a value you specify — when the denominator is zero, instead of throwing a division error that shows up as a red error box on a card visual. That matters a lot in a live dashboard, because a stakeholder can filter down to a segment with zero orders, and I don't want Average Order Value to break the whole visual when that happens.

Q: Why DISTINCTCOUNT for Total Orders and Total Customers instead of COUNT?

A: COUNT would count every row, which is fine only if OrderID or CustomerID never repeats in that table. DISTINCTCOUNT protects the measure even if the grain changes upstream — for example, if Orders ever got a duplicate row from an ETL hiccup, DISTINCTCOUNT still reports the correct number of unique orders.

Q: How would you add a Year-over-Year growth measure?

```
Sales Prior Year =
CALCULATE([Total Sales], SAMEPERIODLASTYEAR(Calendar[Date]))

Sales YoY % =
DIVIDE([Total Sales] - [Sales Prior Year], [Sales Prior Year])
```

A: This requires the Calendar table to be marked as an official Date table first, so SAMEPERIODLASTYEAR can correctly shift the date context back one year. I've documented this as a recommended next-phase measure rather than claiming it's already built — it's important to be precise about what's implemented versus what's planned.

Q: What would you do if two measures gave conflicting numbers for 'Total Sales' in different visuals?

A: First check if they're actually referencing the same underlying measure or if someone built a second, slightly different calculation directly on a visual instead of using the shared measure. That's exactly why I centralized everything into one KPI Measures table — to prevent that kind of drift. If it still disagreed, I'd check filter context differences between the two visuals — a slicer or visual-level filter one visual has that the other doesn't is the most common cause.

5. Power Query & Data Cleaning Questions

Q: What did your data cleaning process actually involve?

A: I ran each table through a standard quality checklist in Power Query: verified primary keys were actually unique, corrected data types (dates as Date, prices as Decimal), standardized text fields like State, City, Category, and PaymentMode so the same value wasn't stored two different ways, and removed blank or invalid rows. I did this in Power Query rather than DAX because it's cheaper to fix data shape once at load time than to keep working around it in every measure.

Q: Why generate the Calendar table with DAX instead of importing it?

A: Generating it guarantees a complete, gap-free sequence of dates that matches my reporting range exactly, rather than depending on whatever dates happen to already exist in the Orders table. That matters for trend visuals — if a month has zero orders, I still want that month to appear on the axis rather than silently disappearing.

Q: How would you validate that your Power Query transformations didn't silently drop or corrupt rows?

A: Row-count checks before and after each major step, and spot-checking aggregate totals — like total Net Sales — against a source-side calculation, such as a quick SQL SUM, before and after the transformation. If those two numbers don't reconcile, something in the ETL step changed data it shouldn't have.

6. Dashboard Design & UX Questions

Q: Why did you choose this particular layout?

A: I followed the F-pattern reading model — KPI cards up top since that's the first thing an executive's eyes land on, trend analysis next since that answers the natural follow-up question of 'is this improving,' and then regional, category, and payment breakdowns lower down for anyone who wants to dig into 'why.' It's a deliberate ordering from headline to context to detail, not just an arrangement of whatever visuals I had.

Q: Why put all the slicers on the left rail instead of at the top?

A: Consistency and discoverability — a user always knows exactly where to look to see or change what the report is filtered to, instead of hunting across the canvas. It also keeps the main content area uncluttered for the actual analytical visuals.

Q: How do you know this dashboard is actually usable, not just functional?

A: I'd want to validate that with real stakeholder walkthroughs and, ideally, simple task-based testing — can a Sales Manager answer 'which region underperformed last quarter' in under 30 seconds without help. In this project's scope, usability was guided by established UX heuristics like the F-pattern and progressive disclosure, but I'd treat stakeholder feedback as the real test, not my own judgment alone.

7. Tough & Skeptical Mentor Questions

These are the harder, more pointed questions a mentor or senior interviewer is likely to ask specifically to probe whether you actually understand your own decisions, or just followed a tutorial.

Q: Why should I believe you understand this model and didn't just copy a tutorial's structure?

A: I can walk through why each relationship is single-direction instead of bidirectional, why DISTINCTCOUNT matters over COUNT specifically for this Orders table, and why I rejected both a flat table and a snowflake schema for this data — those are decisions with tradeoffs, not steps in a tutorial. I'm also upfront in my documentation about what's out of scope, like row-level security and Time Intelligence, which is the kind of thing you only note if you're thinking about the model critically rather than just finishing a checklist.

Q: This is a small, clean dataset. What breaks if I hand you a messy, real 10-million-row retail extract instead?

A: Several things I'd need to handle explicitly: duplicate primary keys that DISTINCTCOUNT would mask rather than fix, inconsistent category naming that needs a proper mapping table instead of just trimming whitespace, and refresh performance — I'd likely need incremental refresh and possibly move slicers-heavy visuals to aggregation tables. I'd also expect null handling in Power Query to become a much bigger job than it was here.

Q: Why should this be in Power BI at all instead of just a SQL dashboard or a Python/Streamlit app?

A: Power BI's strength here is putting a governed, interactive, drag-and-drop self-service tool directly in front of non-technical business stakeholders without needing them to touch code, and DAX gives me the same kind of business-logic centralization SQL views would, but with native visual interactivity and cross-filtering built in. If the requirement were closer to a custom internal tool with heavy custom logic or ML integration, I'd agree a coded app might fit better — this project's scope was standard executive BI reporting, which is squarely Power BI's use case.

Q: What's the weakest part of this project, honestly?

A: Right now there's no row-level security, so anyone with the report sees all regions and all customers — fine for a single-team internal demo, not fine for a real multi-region rollout. I've documented that as a known limitation rather than pretending it's already solved, and it's the first thing I'd build next.

Q: How would you prove this dashboard actually improved decision-making, not just that it looks good?

A: I'd want to track adoption and time-to-insight metrics after rollout — for example, how long it used to take to answer 'how did Region West do last quarter' versus now, and whether report usage logs in Power BI Service show stakeholders actually opening it regularly rather than reverting to their old spreadsheet. Without that follow-up data, the honest answer is that this project demonstrates capability, not yet measured business impact.

8. Behavioral Questions (STAR Method)

Use the Situation–Task–Action–Result structure. Below are suggested angles using this project as your example — adapt the wording to sound like you, not like a script.

Q: Tell me about a time you had to make a design decision with a tradeoff.

A: Situation: choosing between a star schema, snowflake schema, and a flat table for the retail model. Task: pick a structure that would stay simple to maintain and perform well as data grew. Action: I evaluated each option against my actual dimensions — none had repeating attribute groups that justified snowflaking — and chose star schema with single-direction relationships. Result: every DAX measure stayed simple (no complex CALCULATE gymnastics to work around ambiguous filter paths), and I documented the rejected alternatives so the reasoning is auditable, not just assumed.

Q: Tell me about a time you had to simplify something complex for a non-technical audience.

A: Situation: the dashboard needed to serve a CEO, Sales Manager, Marketing Manager, Finance, and Operations — each with a different priority. Task: design one page that served all of them without overwhelming any of them. Action: I mapped each stakeholder to one headline KPI and used an F-pattern layout so no one had to hunt for their number. Result: a single-page report that answers five different roles' first question within seconds, rather than five separate reports.

Q: Describe a time you caught a mistake before it became a bigger problem.

A: Situation: while validating Order_Details, I checked whether Net Sales actually reconciled with Gross Sales minus Discount Amount across the table. Task: confirm the discount logic was internally consistent before building any KPI on top of it. Action: I ran a validation check comparing calculated versus stored Net Sales values in Power Query. Result: caught it at the data-quality stage rather than after building a Total Sales measure on top of a silently wrong column, which is exactly the kind of bug that erodes stakeholder trust once discovered downstream.

9. Quick-Reference Cheat Sheet

Core Measures

Measure	Formula
Total Sales	SUM(Order_Details[Net Sales])
Total Orders	DISTINCTCOUNT(Orders[OrderID])
Total Customers	DISTINCTCOUNT(Customers[CustomerID])
Total Quantity Sold	SUM(Order_Details[Quantity])
Average Order Value	DIVIDE([Total Sales], [Total Orders])

Key Numbers to Remember

Metric	Value
Total Sales	1.72bn
Total Orders	5,000
Total Customers	500
Total Quantity Sold	45K
Average Order Value	344.81K
Top Category	Furniture (~26.7% of sales)
Top States by Customer Count	Maharashtra, Delhi, Tamil Nadu

Relationships

From	To	Cardinality
Customers.CustomerID	Orders.CustomerID	One-to-Many
Orders.OrderID	Order_Details.OrderID	One-to-Many
Products.ProductID	Order_Details.ProductID	One-to-Many
Calendar.Date	Orders.OrderDate	One-to-Many

If You Freeze, Fall Back To This

If a question catches you off guard, it's fine to say: "Let me think through that for a second" — pause, then reason out loud from what you know is true: the model has four dimension tables and one fact table, every KPI comes from one

centralized measure table, and every design choice was made to keep filter context predictable. Reasoning from those anchors will get you to a sensible answer even for a question you haven't specifically rehearsed.